

Nama: Julius Satria Agung

NRP: 152024049 (Uneven for dynamic distribution)

Kelas: BB

- **Static distribution**

Static distribution adalah metode di mana pembagian task dilakukan sekali di awal, sebelum eksekusi dimulai, dan tidak berubah selama proses berjalan.

Cara Kerja

1. Total task diketahui di awal
2. Task dibagi ke worker secara langsung (biasanya rata)
3. Setiap worker mengerjakan bagiannya masing-masing
4. Tidak ada komunikasi ulang antar worker

- **Dynamic distribution**

Dynamic distribution adalah metode di mana task tidak dibagi di awal, tetapi diberikan ke worker secara bertahap saat runtime.

Cara Kerja

1. Semua task dimasukkan ke dalam queue
2. Worker mengambil task satu per satu
3. Setelah selesai → langsung ambil task berikutnya
4. Berjalan sampai queue kosong

Code:

```
# Nama: Julius Satria Agung
# NRP: 152024049
# Kelas: BB
import time
import random
from multiprocessing import Process, Queue

def task_execution(task_id):
```

```
duration = random.uniform(0.5, 2.0)
time.sleep(duration)
return (task_id, duration)
```

```
def static_worker(tasks, result_queue):
```

```
    total_time = 0
    for task in tasks:
        start = time.time()
        _, duration = task_execution(task)
        total_time += duration
    result_queue.put(total_time)
```

```
def static_distribution(num_tasks, num_workers):
```

```
    tasks = list(range(num_tasks))
    chunk_size = num_tasks // num_workers
```

```
    result_queue = Queue()
    processes = []
```

```
    start_time = time.time()
```

```
    for i in range(num_workers):
```

```
        chunk = tasks[i * chunk_size:(i + 1) * chunk_size]
        p = Process(target=static_worker, args=(chunk, result_queue))
        p.start()
        processes.append(p)
```

```
    for p in processes:
```

```
        p.join()
```

```
total_execution = time.time() - start_time
```

```
print("\n[STATIC DISTRIBUTION]")
```

```
print("Ideal distribution tercapai jika semua worker selesai hampir bersamaan.")
```

```
print(f"Total waktu eksekusi: {total_execution:.2f} detik")
```

```
def dynamic_worker(task_queue, result_queue):
```

```
    while not task_queue.empty():
```

```
        try:
```

```
            task = task_queue.get_nowait()
```

```
        except:
```

```
            break
```

```
        start = time.time()
```

```
        _, duration = task_execution(task)
```

```
        result_queue.put(duration)
```

```
def dynamic_distribution(num_tasks, num_workers):
```

```
    task_queue = Queue()
```

```
    result_queue = Queue()
```

```
    for i in range(num_tasks):
```

```
        task_queue.put(i)
```

```
    processes = []
```

```
    start_time = time.time()
```

```
    for _ in range(num_workers):
```

```
        p = Process(target=dynamic_worker, args=(task_queue, result_queue))
```

```
        p.start()
```

```

        processes.append(p)

for p in processes:
    p.join()

total_execution = time.time() - start_time

print("\n[DYNAMIC DISTRIBUTION]")
print("Optimal time tercapai saat semua worker selalu sibuk tanpa idle.")
print(f"Total waktu eksekusi: {total_execution:.2f} detik")

if __name__ == "__main__":
    NUM_TASKS = 20
    NUM_WORKERS = 4

    static_distribution(NUM_TASKS, NUM_WORKERS)
    dynamic_distribution(NUM_TASKS, NUM_WORKERS)

```

STATIC DISTRIBUTION — Alur Eksekusi

Step-by-step runtime:

1. Generate task

```
tasks = list(range(num_tasks))
```

→ menghasilkan: [0,1,2,...,19]

2. Bagi task ke worker

```
chunk_size = num_tasks // num_workers
```

→ $20 / 4 = 5$ task per worker

3. Buat process

```
p = Process(target=static_worker, args=(chunk, result_queue))
```

→ Setiap worker langsung dapat:

- Worker 1 → task 0–4
- Worker 2 → task 5–9
- dst

4. Worker mulai eksekusi

```
def static_worker(tasks, result_queue):
```

Loop:

for task in tasks:

Eksekusi:

- Ambil task
- Jalankan task_execution()
- Tambahkan durasi ke total_time

5. Tidak ada redistribusi

- Worker tidak bisa ambil task dari worker lain
- Kalau cepat → idle
- Kalau lambat → jadi bottleneck

6. Sinkronisasi

```
p.join()
```

DYNAMIC DISTRIBUTION — Alur Eksekusi

Step-by-step runtime:

1. Masukkan semua task ke queue

```
task_queue.put(i)
```

→ Tidak dibagi di awal

2. Worker dibuat

`Process(target=dynamic_worker, ...)`

Semua worker start tanpa pembagian tetap

3. Worker ambil task

`task = task_queue.get_nowait()`

Ini kunci dynamic:

- Worker ambil task saat siap
- Tidak ada assignment tetap

4. Loop eksekusi

`while not task_queue.empty():`

Alurnya:

1. Ambil task
2. Eksekusi
3. Simpan hasil
4. Ambil lagi

5. Load balancing otomatis

Yang terjadi di runtime:

- Worker cepat → ambil lebih banyak task
- Worker lambat → ambil lebih sedikit
- Semua worker tetap aktif selama task masih ada

6. Sinkronisasi

`p.join()`

→ Tunggu semua task selesai

```
● PS C:\Kuliah\Semester 4\Komputasi Paralel dan Sistem Distribusi\quiz> & C:/Users/...  
r 4/Komputasi Paralel dan Sistem Distribusi/quiz/dynamic_dist.py"  
  
[STATIC DISTRIBUTION]  
Ideal distribution tercapai jika semua worker selesai hampir bersamaan.  
Total waktu eksekusi: 7.72 detik  
  
[DYNAMIC DISTRIBUTION]  
Optimal time tercapai saat semua worker selalu sibuk tanpa idle.  
Total waktu eksekusi: 6.98 detik
```

Analysis:

Dynamic distribution lebih efisien untuk uneven workload karena mampu menyesuaikan distribusi task secara real-time. Karena worker yang udah beres tasknya akan membantu task worker lainnya sehingga mencapai total waktu eksekusi yang minimum.

Dibandingkan static distribution karena memaksakan meratakan worker sesuai loadnya, tanpa melihat performa workernya, sehingga ada yang worker beres duluan, lalu idle dan ada worker yang belum selesai, ini yang disebut work load imbalance.